Handwritten Digits Recognition Use-case

27 July 2025 21:07

Using TensorFlow Keras

MNIST Database

↓

Modified National Institute of Standards and Technology Database

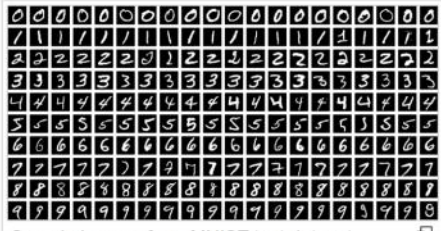
MNIST database

14 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

The **MNIST database** (*Modified National Institute of Standards and Technology database*[1]) is a large database of handwritten digits that is commonly used for training various image processing systems.[2][3] The database is also widely used for training and testing in the field of machine learning.[4][5] It was created by "re-mixing" the samples from NIST's original datasets.[6] The creators felt that since NIST's training dataset was taken from American Census Bureau employees, while the testing dataset was taken from American high school students, it was not well-suited for machine learning experiments.[7] Furthermore, the black and white images from NIST were normalized to fit into a 28x28 pixel bounding box and anti-aliased, which introduced grayscale levels.[7]



Sample images from MNIST test dataset

The MNIST database contains 60,000 training images and 10,000 testing images.[8] Half of the training set and half of the test set were taken from NIST's training dataset, while the other half of the training set and the other half of the test set were taken from NIST's testing dataset.[9] The original creators of the database keep a list of some of the methods tested on it.[7] In their original paper, they use a support-vector machine to get an error rate of 0.8%.[10]

The original MNIST dataset contains at least 4 wrong labels.[11]

Training dataset	#	60,000 images
Testing dataset	#	10,000 images
Total	#	70,000 images

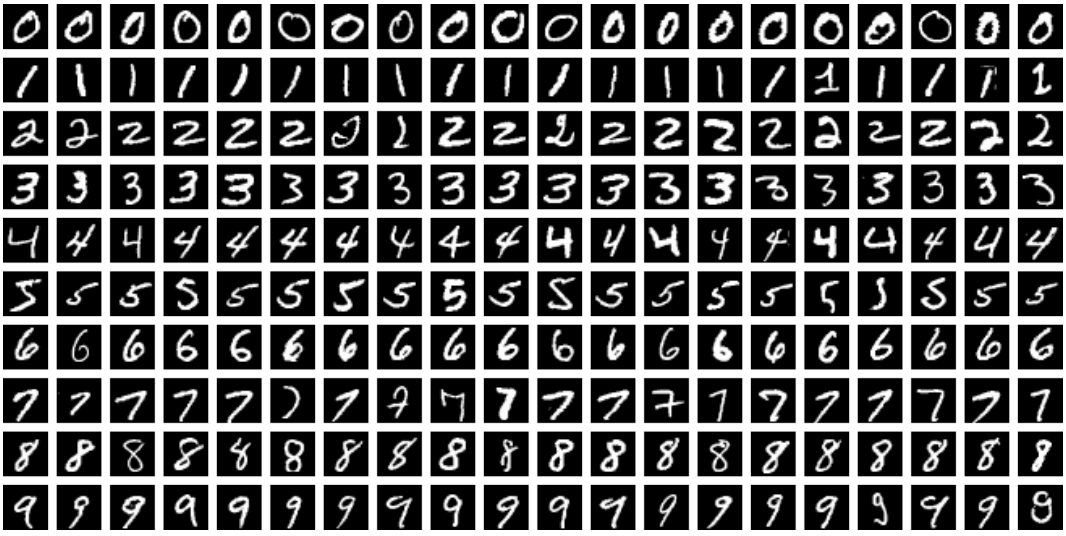
Testing Proportion % = 10000/70000 = 0.1429 *100 = ***14.29 %***

To get a custom train & test split

step # ① Merge train and test dataset along with labels [0-9]

step #② Use train_test_split from scikit learn and get test dataset of the appropriate test size.

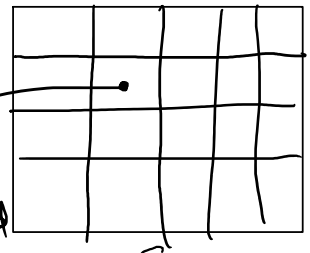
Step #(2) Use train_test_split from scikit learn and get test dataset of the appropriate test size.



Note: 70K rows → 70K images of handwritten digits

↓

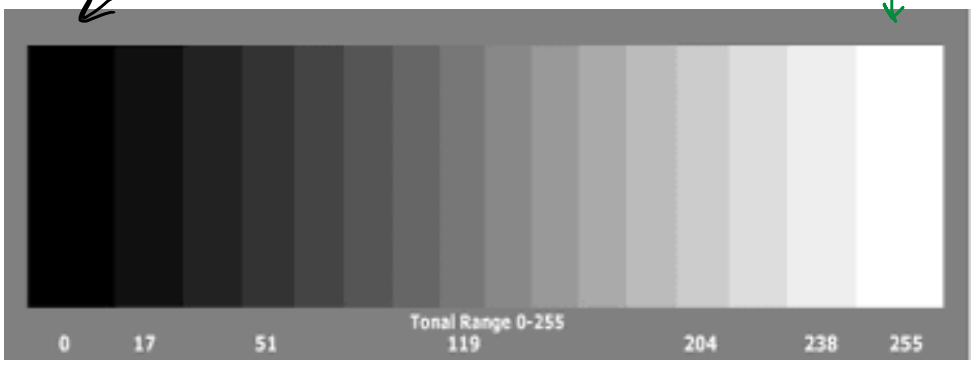
each of these images has already been converted into 'pixelated form'



each pixel has a value assigned from the grayscale range [0-255]

dark black

bright white



Total combinations # $256 = 2^8$

Total combinations # $256 = 2^8$

We'll be using MNIST datasets via TF datasets

https://www.tensorflow.org/datasets/catalog/mnist

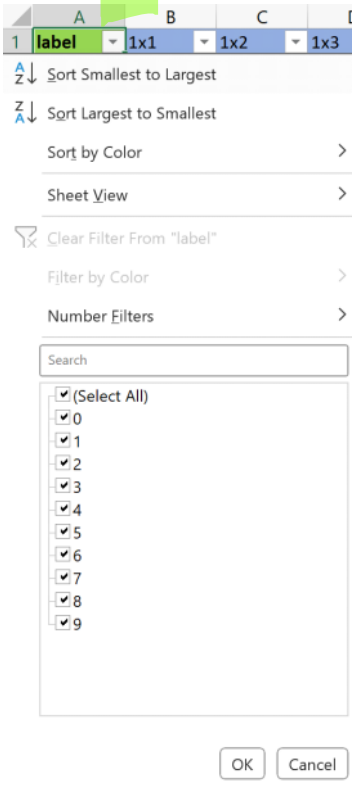
Split	Examples
'test'	10,000
'train'	60,000

- **Feature structure:**

```
FeaturesDict({
    'image': Image(shape=(28, 28, 1), dtype=uint8),
    'label': ClassLabel(shape=(), dtype=int64, num_classes=10),
})
```

- **Feature documentation:**

Feature	Class	Shape	Dtype	Description
	FeaturesDict			
image	Image	(28, 28, 1)	uint8	
label	ClassLabel		int64	0 1 2 3 4 5 6 7 8 9



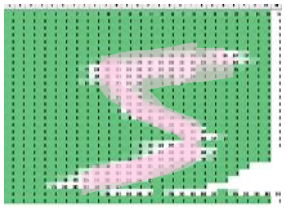
[4x4 array]

	1	2	3	4
1	1x1	1x2	1x3	1x4
2				
3		3x2	3x3	
4				4x4

16 cells

16 features

actual data → 784 columns



Greyscale range [0-255]

28 columns

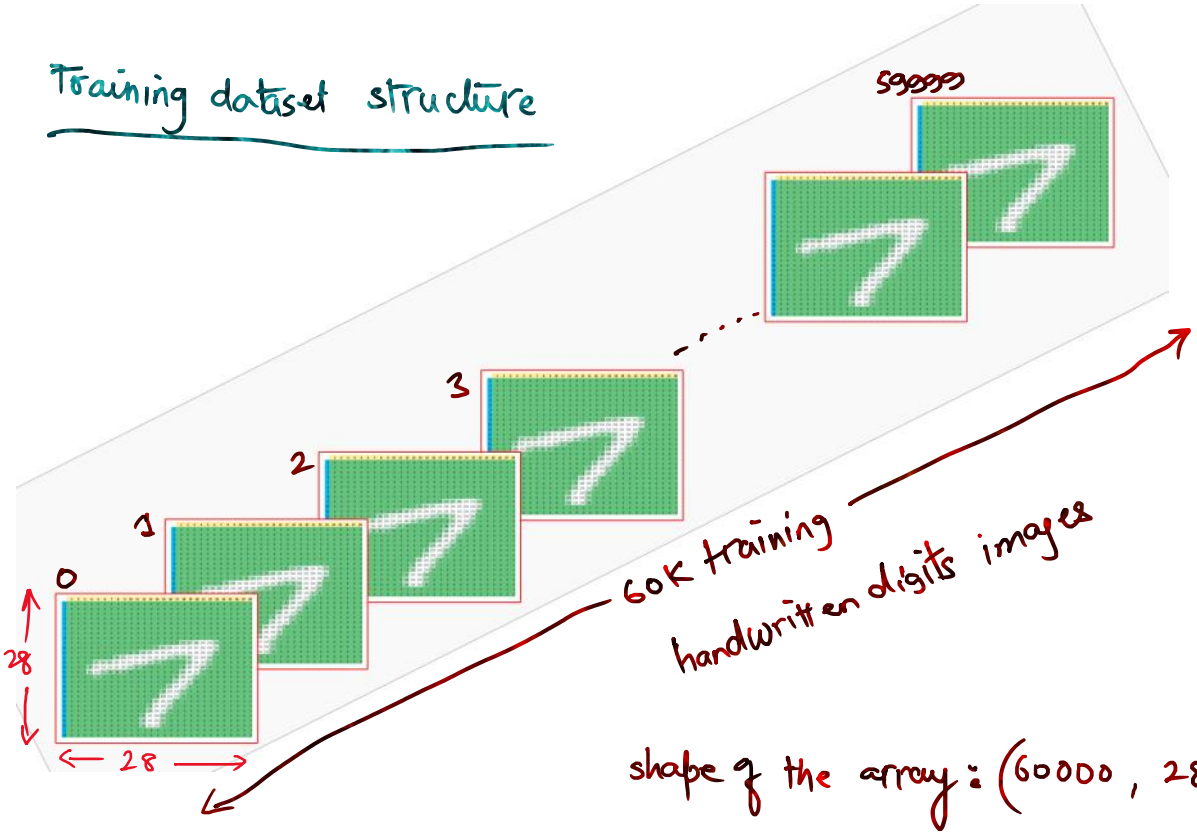
28 Rows

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
1	48	253	138	194	90	90	110	188	55	13	146	77	40	51	76	132	69	134	154	227	69	240	191	125	231	168	206	17
2	35	251	17	226	141	84	109	119	178	194	209	153	13	29	14	177	2	200	51	188	52	126	208	119	127	16	37	218
3	104	3	220	205	175	221	140	235	112	120	30	83	38	179	69	114	112	57	67	39	88	6	1	133	123	114	83	45
4	134	156	182	53	214	144	154	173	97	6	14	52	211	84	137	91	31	177	178	124	21	143	186	197	145	205	4	58
5	211	250	104	116	142	51	24	20	17	132	221	170	35	13	199	151	95	137	111	229	227	121	97	40	162	234	212	182
6	11	181	168	188	117	59	218	148	190	222	107	232	145	76	29	216	215	28	160	59	106	138	182	181	191	191	44	48
7	152	26	116	74	101	79	177	252	180	164	38	255	228	76	200	218	210	217	167	58	193	40	167	48	25	33	218	112
8	159	248	98	44	97	109	179	101	172	181	133	37	78	36	163	98	28	247	124	170	147	140	149	250	102	171	249	11
9	51	92	238	153	201	84	228	219	163	65	233	179	223	151	9	173	99	51	84	216	57	156	61	15	239	106	149	209
10	106	118	189	46	75	51	48	243	115	162	165	235	11	153	65	115	251	221	204	200	253	237	232	109	113	89	50	196
11	241	207	123	223	169	52	213	191	15	181	44	57	29	47	136	19	204	120	12	3	191	183	213	67	22	125	69	177
12	77	74	103	49	96	29	226	230	213	38	186	252	80	245	149	36	33	34	211	7	126	73	96	106	208	175	38	127
13	20	36	37	223	178	224	207	60	121	252	30	124	147	137	133	159	174	253	183	76	79	150	169	208	45	102	245	22
14	89	59	72	159	84	191	48	156	241	165	1	158	184	53	7	186	76	78	0	13	29	145	179	157	123	114	24	68
15	130	144	45	114	8	151	189	166	94	204	160	71	3	72	17	129	89	17	214	132	184	195	94	200	141	1	138	3
16	225	240	203	15	69	105	53	5	254	71	10	157	194	235	228	219	137	254	218	39	230	233	65	4	137	104	164	76
17	185	92	11	65	253	8	8	69	221	21	75	99	234	248	213	225	85	251	85	248	62	254	80	212	203	89	244	184
18	180	197	45	58	151	56	205	87	242	232	44	128	247	52	203	143	236	26	30	87	145	247	20	98	148	130	219	183
19	116	16	249	77	0	132	89	62	69	179	249	90	145	229	179	141	50	23	193	148	104	81	156	245	132	176	88	120
20	54	232	153	41	199	111	181	148	140	51	194	63	107	211	196	253	227	146	243	233	197	225	145	251	223	85	151	245
21	0	189	230	216	138	46	107	234	100	228	180	44	80	98	203	108	251	128	212	73	114	236	172	233	252	71	241	93
22	225	0	113	209	10	105	178	118	179	224	90	203	136	245	215	17	211	203	42	128	199	86	190	211	21	108	99	19
23	147	27	160	187	73	244	127	3	203	38	72	77	20	12	81	186	24	140	109	237	191	242	83	181	22	156	46	121
24	201	206	175	168	248	126	229	94	212	40	71	24	211	63	211	17	211	12	123	163	17	251	83	19	212	209	128	137
25	116	143	195	77	216	240	234	199	100	167	164	168	10	56	240	71	215	56	244	187	247	105	198	28	76	131	10	37
26	197	123	200	150	4	59	168	157	179	34	22	212	113	167	228	18	75	88	73	193	10	132	65	142	131	62	0	52
27	106	34	183	90	134	218	50	1	148	110	114	78	65	187	213	157	55	32	157	47	234	78	192	210	107	91	153	22
28	189	228	21	228	41	46	116	241	89	167	10	240	74	57	213	2	242	108	250	33	104	229	191	174	59	15	16	127

digit no # 7.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	7	7	4	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4	13	120	137	186	254	254	136	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0	5	53	143	179	254	254	254	255	254	254	193	0	0	0	0	0
11	0	0	0	0	0	0	0	0	4	25	50	149	175	254	254	254	254	254	254	255	254	254	108	0	0	0	0	0
12	0	0	0	0	0	0	0	96	170	254	254	254	254	254	245	183	99	99	147	252	254	196	3	0	0	0	0	0
13	0	0	0	4	145	162	162	219	254	254	254	246	186	93	66	0	0	0	125	246	254	119	2	0	0	0	0	0
14	0	0	0	87	215	254	254	254	254	204	87	68	0	0	0	0	0	46	197	254	180	47	0	0	0	0	0	0
15	0	0	0	168	254	156	80	80	80	28	0	0	0	0	0	0	0	126	245	255	57	0	0	0	0	0	0	0
16	0	0	0	2	74	33	0	0	0	0	0	0	0	0	0	0	43	200	254	91	6	0	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	156	245	205	13	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	124	244	254	85	6	0	0	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	59	217	254	121	7	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	176	254	254	61	0	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0	0	0	0	0	161	245	215	58	7	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0	0	0	0	0	54	246	254	185	7	0	0	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0	0	0	0	0	124	254	254	64	0	0	0	0	0	0	0	0	0	0	0	0
24	0	0	0	0	0	0	0	0	0	0	0	52	237	254	126	5	0	0	0	0	0	0	0	0	0	0	0	0
25	0	0	0	0	0	0	0	0	0	0	0	118	254	254	119	0	0	0	0	0	0	0	0	0	0	0	0	0
26	0	0	0	0	0	0	0	0	0	0	0	118	254	254	61	0	0	0	0	0	0	0	0	0	0	0	0	0
27	0	0	0	0	0	0	0	0	0	0	0	2	6	6	2	0	0	0	0	0	0	0	0	0	0	0	0	0
28	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

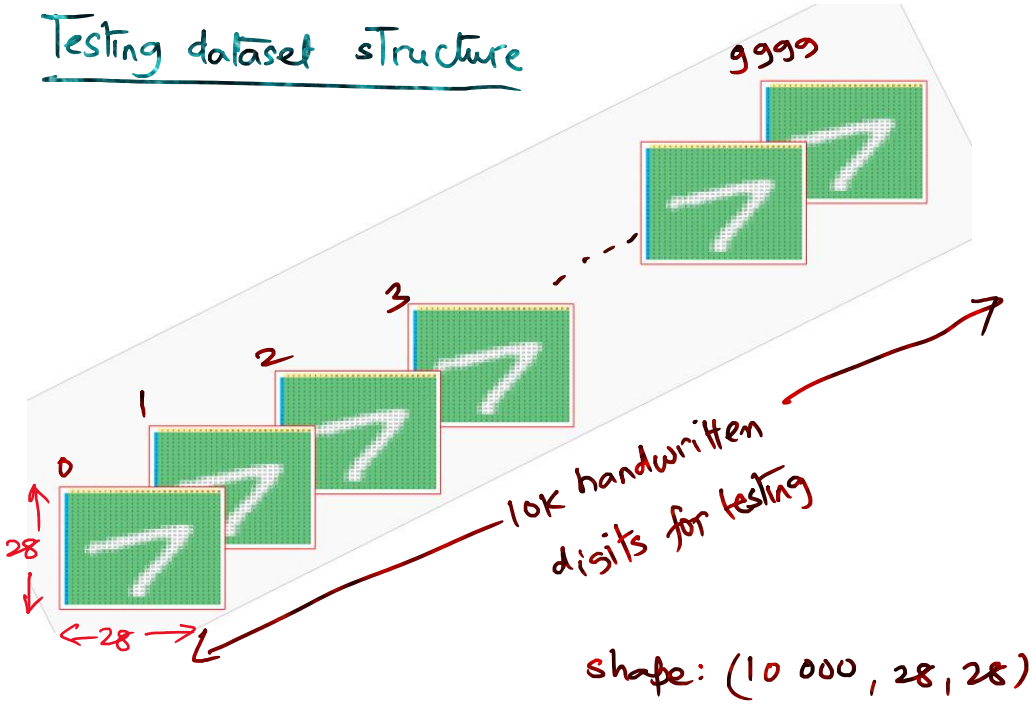
Training dataset structure



shape of the array: (60000, 28, 28)

Note: Not all images are for '7' → This is just an illustration

Testing dataset structure



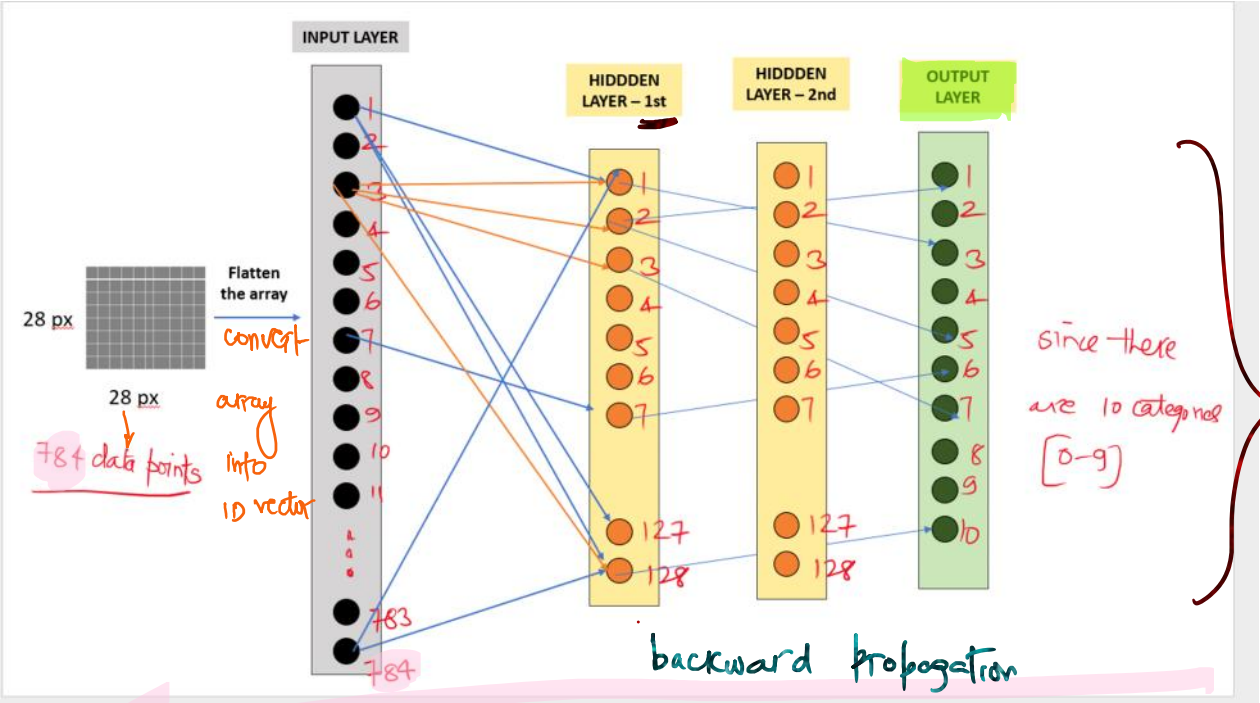
shape: (10 000, 28, 28)

BUILDING THE MODEL

MNIST Handwritten digits NN model's architecture

Input Layer → 2 Hidden Layers → output Layer

Forward →



Compute loss & accuracy

784 128 128 10